Chapter 1. Defining a Discipline

The most important thing is to find out what is the most important thing.

—Shunryu Suzuki

Software security—the idea of engineering software so that it continues to function correctly under malicious attack—is not really new, but it has received renewed interest over the last several years as reactive network-based security approaches such as firewalls have proven to be ineffective. Unfortunately, today's software is riddled with both design flaws and implementation bugs, resulting in unacceptable security risk. As Cheswick and Bellovin put it, "any program, no matter how innocuous it seems, can harbor security holes" [Cheswick and Bellovin 1994]. The notion of software security risk has become common knowledge, yet developers, architects, and computer scientists have only recently begun to systematically study how to build secure software.

The network security market weighs in at around $45 billion.[1] However, the 532% increase in CERT incidents reported (2000–2003)[2] and the fact that 43% of 500 companies responding to a popular e-crime survey reported an increase in cybercrime[3] show that whatever we're doing is clearly not working. Basically, the dollars spent on network security and other perimeter solutions are not solving the security problem. We must build better software.

[1] Network security total market value as reported by the analyst firm IDC in February 2003, Worldwide Security Market <http://www.idc.com/getdoc.jsp?containerId=32391>.

[2] According to data from Carnegie Mellon University's (CMU) Software Engineering Institute's (SEI) CERT Coordination Center (shown in Figure 1-1) <http://www.cert.org>.

[3] E-Crime Watch Survey, 2004. Sponsored by CSO Magazine, United States Secret Service, and CMU SEI CERT Coordination Center <http://www.csoonline.com/releases/ecrimewatch04.pdf>.

A body of software security literature has begun to emerge in the research community, but in practical terms the practice of software security remains in its infancy.[4] The first books on software security and security engineering, for example, were published as recently as 2001 [Anderson 2001; Viega and McGraw 2001; Howard and LeBlanc 2002]. Today, a number of references do a good job of providing a philosophical underpinning for software security and discussion of particular technical issues, but much remains to be done to put software security into practice. This book is designed to help.

[4] See Chapter 13 for annotated pointers into the software security literature.

The Security Problem

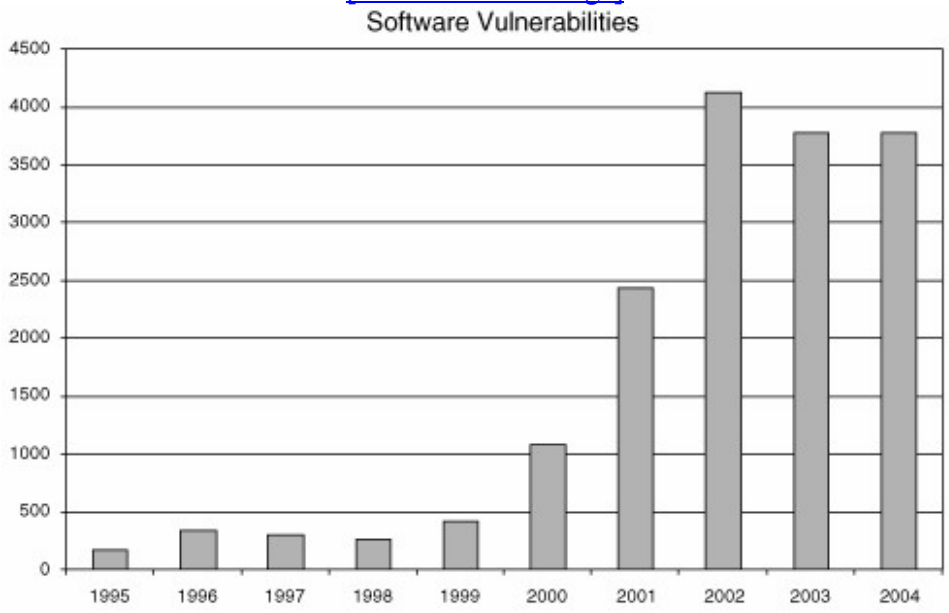
A central and critical aspect of the computer security problem is a software problem. Software defects with security ramifications—including implementation bugs such as buffer overflows and design flaws such as inconsistent error handling—promise to be with us for years. All too often malicious intruders can hack into systems by exploiting software defects [Hoglund and McGraw 2004]. Moreover, Internet-enabled software applications are a commonly (and too easily) exploited target, with software's ever-increasing complexity and extensibility adding further fuel to the fire. By any measure, security holes in software are common, and the problem is growing.

The security of computer systems and networks has become increasingly limited by the quality and security of the software running on constituent machines. Internet-enabled software, especially custom applications that use the Web, are a sadly common target for attack. Security researchers and academics estimate that more than half of all vulnerabilities are due to buffer overruns, an embarrassingly elementary class of bugs [Wagner et al. 2000]. Of course, more complex problems, such as race conditions and design errors, wait in the wings for the demise of the buffer overflow. These more subtle (but equally dangerous) kinds of security problems appear to be just as prevalent as simple bugs.

Security holes in software are common. Over the last five years the problem has grown. Figure 1-1 shows the number of security-related software vulnerabilities reported to the CERT Coordination Center (CERT/CC) from 1995 through 2004. There is a clear and pressing need to change the way we approach computer security and to develop a disciplined approach to software security.

Figure 1-1. The number of security-related software vulnerabilities reported to CERT/CC over several years. Though the widespread adoption of network security technology continues, the problem persists.

[View full size image]



Software security is about understanding software-induced security risks and how to manage them. Good software security practice leverages good software engineering practice and involves thinking about security early in the software lifecycle, knowing and understanding common problems (including language-based flaws and pitfalls), designing for security, and subjecting all software artifacts to thorough objective risk analyses and testing. As you can imagine, software security is a knowledge-intensive field.

Software is everywhere. It runs your car. It controls your cell phone. It keeps your dishwasher going. It is the lifeblood of your bank and the nation's power grid. And sometimes it even runs on your computer. What's important is realizing just how widespread software is. As businesses and society come to depend more heavily on software, we have to make it better. Now that software is networked by default, software security is no longer a luxury—it's a necessity.

The Trinity of Trouble: Why the Problem Is Growing

Most modern computing systems are susceptible to software security problems, so why is software security a bigger problem now than in the past? Three trends—together making up the trinity of trouble—have a large influence on the growth and evolution of the problem.[5]

[5] Interestingly, these three general trends are also responsible for the alarming rise of malicious code [McGraw and Morrisett 2000].

Connectivity

The growing connectivity of computers through the Internet has increased both the number of attack vectors and the ease with which an attack can be made. This puts software at greater risk. More and more computers, ranging from home PCs to systems that control critical infrastructure, such as the supervisory control and data acquisition (SCADA) systems that run the power grid, are being connected to enterprise networks and to the Internet. Furthermore, people, businesses, and governments are increasingly dependent on network-enabled communication such as e-mail or Web pages provided by information systems. Things that used to happen offline now happen online. Unfortunately, as these systems are connected to the Internet, they become vulnerable to software-based attacks from distant sources. An attacker no longer needs physical access to a system to exploit vulnerable software; and today, software security problems can shut down banking services and airlines (as shown by the SQL Slammer worm of January 2003).

Because access through a network does not require human intervention, launching automated attacks is easy. The ubiquity of networking means that there are more software systems to attack, more attacks, and greater risks from poor software security practices

than in the past. We're really only now beginning to cope with the ten-year-old attack paradigm that results from poor coding and design. Ubiquitous networking and attacks directly related to distributed computation remain rare (though the network itself is the primary vector for getting to and exploiting poor coding and design problems). This will change for the worse over time. Because the Internet is everywhere, the attackers are now at your virtual doorstep.

To make matters worse, large enterprises have caught two bugs: Web Services and its closely aligned Service Oriented Architecture (SOA). Even though SOA is certainly a fad driven by clever marketing, it represents a succinct way to talk about what many security professionals have always known to be true: Legacy applications that were never intended to be inter-networked are becoming inter-networked and published as services.

Common platforms being integrated into megasolutions include SAP, PeopleSoft, Oracle, Informatica, Maestro, and so on (not to mention more modern J2EE and .NET apps), COBOL, and other ancient mainframe platforms. Many of these applications and legacy systems don't support common toolkits like SSL, standard plug-ins for authentication/authorization in a connected situation, or even simple cipher use. They don't have the built-in capability to hook into directory services, which most large shops use for authentication and authorization. Middleware vendors pledge they can completely carve out the complexity of integration and provide seamless connectivity, but even though they provide connectivity (through JCA, WBI, or whatever), the authentication and application-level protocols don't align.

Thus, middleware integration in reality reduces to something ad hoc like cross-enterprise FTP between applications. What's worse is that lines of business often fear tight integration with better tools (because they lack skills, project budget, or faith in their infrastructure team), so they end up using middleware to FTP and drop data globs that have to be mopped up and transmogrified into load files or other application input. Because of this issue, legacy product integrations often suffer from two huge security problems:

1. Exclusive reliance on host-to-host authentication with weak passwords
2. Looming data compliance implications having to do with user privacy (because unencrypted transport of data over middleware and the middleware's implementation for failover and load balancing means that queue cache files get stashed all over the place in plain text)

Current trends in enterprise architecture make connectivity problems more problematic than ever before.

Extensibility

A second trend negatively affecting software security is the degree to which systems have become extensible. An extensible system accepts updates or extensions, sometimes referred to as mobile code so that the functionality of the system can be evolved in an

incremental fashion [McGraw and Felten 1999]. For example, the plug-in architecture of Web browsers makes it easy to install viewer extensions for new document types as needed. Today's operating systems support extensibility through dynamically loadable device drivers and modules. Today's applications, such as word processors, e-mail clients, spreadsheets, and Web browsers, support extensibility through scripting, controls, components, and applets. The advent of Web Services and SOA, which are built entirely from extensible systems such as J2EE and .NET, brings explicit extensibility to the forefront.

From an economic standpoint, extensible systems are attractive because they provide flexible interfaces that can be adapted through new components. In today's marketplace, it is crucial that software be deployed as rapidly as possible in order to gain market share. Yet the marketplace also demands that applications provide new features with each release. An extensible architecture makes it easy to satisfy both demands by allowing the base application code to be shipped early, with later feature extensions shipped as needed.

Unfortunately, the very nature of extensible systems makes it hard to prevent software vulnerabilities from slipping in as unwanted extensions. Advanced languages and platforms including Sun Microsystems' Java and Microsoft's .NET Framework are making extensibility commonplace.

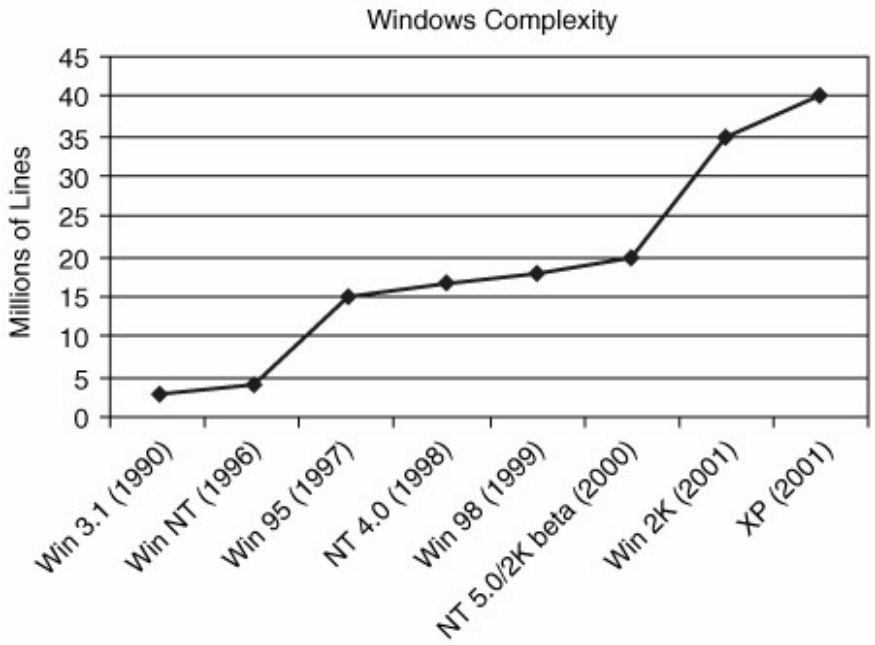
Complexity

A third trend impacting software security is the unbridled growth in the size and complexity of modern information systems, especially software systems. A desktop system running Windows XP and associated applications depends on the proper functioning of the kernel as well as the applications to ensure that vulnerabilities cannot compromise the system. However, Windows XP itself consists of at least forty million lines of code, and end-user applications are becoming equally, if not more, complex. When systems become this large, bugs cannot be avoided.

Figure 1-2 shows how the complexity of Windows (measured in lines of code) has grown over the years. The point of the graph is not to emphasize the numbers themselves, but rather the growth rate over time. In practice, the defect rate tends to go up as the square of code size.[6] Other factors that significantly affect complexity include whether the code is tightly integrated, the overlay of patches and other post-deployment fixes, and critical architectural issues.

[6] See the article "Encapsulation and Optimal Module Size" at <http://www.faqs.org/docs/artu/ch04s01.html#ftn.id2894437>.

Figure 1-2. Growth of the Microsoft operating system code base from 1990 to 2001. These numbers include all aspects of Windows, including device drivers.[7]



[7] With regard to particular names for Microsoft operating systems, see <http://foldoc.doc.ic.ac.uk/foldoc/foldoc.cgi?NT5>.

The complexity problem is exacerbated by the use of unsafe programming languages (e.g., C and C++) that do not protect against simple kinds of attacks, such as buffer overflows. In theory, we could analyze and prove that a small program was free of problems, but this task is impossible for even the simplest desktop systems today, much less the enterprise-wide systems used by businesses or governments.

Of course, Windows is not alone. Almost all code bases tend to grow over time. During the last three years, I have made an informal survey of thousands of developers. With few exceptions (on the order of 1% of sample size), developers overwhelmingly report that their groups intend to produce more code, not less, as time goes by. Ironically, these same developers also report that they intend to produce fewer bugs even as they produce more code. The unfortunate reality is that "more lines, more bugs" is the rule of thumb that tends to be borne out in practice (and in science, as the next section shows). Developers are an optimistic lot.

The propensity for software systems to grow very large quickly is just as apparent in open source systems as it is in Windows (see Table 1-1). The problem is, of course, that more code results in more defects and, in turn, more security risk.

Table 1-1. Source Lines of Code for Major Operating Systems and Kernels

19xx	SCOMP	20,000
1979	Multics	1,000,000
2000	Red Hat 6.2	17,000,000
2000	Debian.GNU/Linux 2.2	55,000,000
2000	Linux 2.2 kernel	1,780,000
2000	XFree86 3.3.6	1,270,000
2001	Red Hat 7.1	30,000,000
2002	Mac OS X Darwin kernel	790,000
Data on this chart gathered by Lee Badger, a DARPA program manager.[8]		

[8] Badger reports the Linux estimate from "Counting Potatoes: The Size of Debian 2.2" by Gonzalez-Barahona et al. <http://people.debian.org/~jgb/debian-counting>, and "More Than a Gigabuck: Estimating GNU/Linux's Size" by David Wheeler. The Multics estimate is from Tom Van Vleck and Charlie Clingen <http://www.multicians.org/mspp.html>.

Sometimes the code base grows (in executable space) even when the source code base appears to be small. Consider what happens when you target the .NET or J2EE platforms. In these situations, you adopt an unfathomably large base of code underneath your application. Things get even worse when you rely on the following:

- Data flattening: Castor, Java Data Objects (JDO), container-managed persistence
- Identity management and provisioning

- XML or other representational formats and associated parsers
- Model View Controller (MVC) frameworks: Struts deployment containers
- Application servers, Web containers
- Databases: Oracle, SQR, Informatica, and so on

To understand what I mean here, you should think about how much bytecode it takes to run "Hello World" in WebSphere or "Hello World" as a Microsoft ASP glob. What exactly is in that 2MB of stuff running on top of the operating system, anyway?

Basic Science

Everyone believes the mantra "more lines, more bugs" when it comes to software, but until recently the connection to security was understood only intuitively. Thanks to security guru Dan Geer, there are now some real numbers to back up this claim. On his never-ending quest to inject science into computer security, Geer has spoken widely

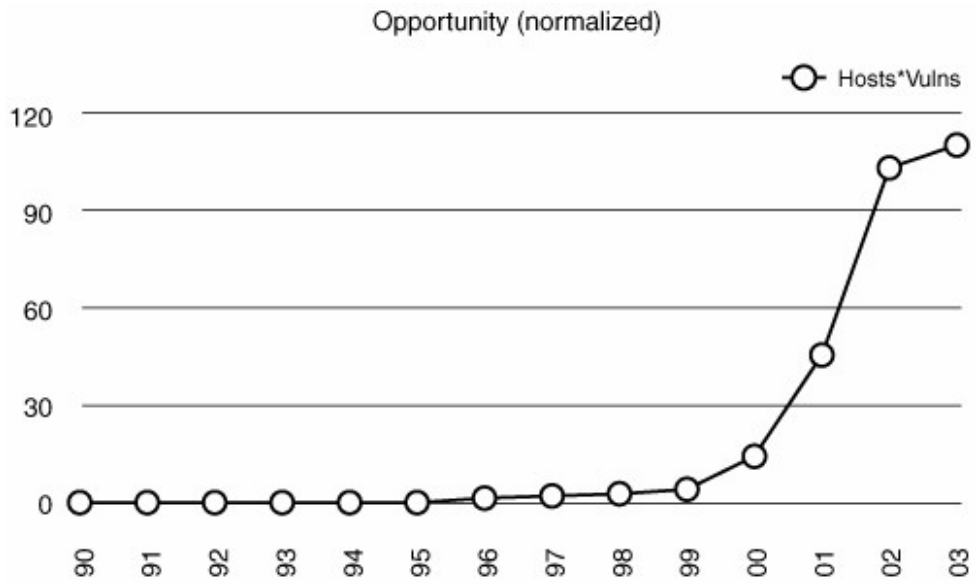
about measurement and metrics. In the now famous monoculture paper, Geer and others decried the (national) security risk inherent in almost complete reliance on buggy Microsoft operating systems (see the acclaimed paper "CyberInsecurity: The Cost of Monopoly" [Geer et al. 2003]). Besides being fired from his job at @stake for the trouble, Geer raised some interesting questions about security bugs and the pile of software we're creating. One central question emerged: Is it true that more buggy code leads to more security problems in the field? What kind of predictive power do we get if we look into the data?

Partially spurred by an intense conversation we had, Geer did some work correlating CERT vulnerability numbers, number of hosts, and lines of code, which he has since presented in several talks. In an address at the Yale Law School,[9] Geer presented some correlations that bear repeating here. If you begin with the CERT data and the lines of code data presented in Figure 1-2 you can then normalize the curves.

[9] Dan Geer, "The Physics of Digital Law," keynote address, CyberCrime and Digital Law Enforcement Conference, Information Society Project, Yale Law School, March 26, 2004. (Unpublished slides.)

Geer describes "opportunity" as the normalized product of the number of hosts (gleaned from publicly available Internet Society data) and the number of vulnerabilities (shown in Figure 1-1). See Figure 1-3. One question to ask is whether there is "untapped opportunity" in the system as understood in this form. Geer argues that there is, by comparing actual incidents curves against opportunity (not shown here). Put simply, there are fewer incidents than there could be. Geer believes that this indicates a growing reservoir of trouble.

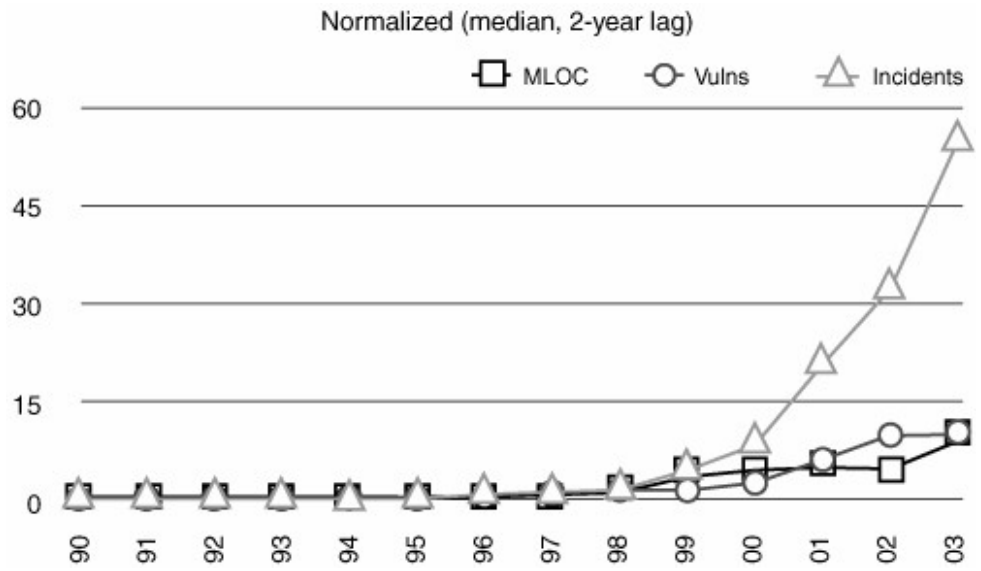
Figure 1-3. Total number of open holes, or "opportunity," as a normalized product of the number of hosts and the number of vulnerabilities (vulns). (After Geer.)



By normalizing the lines-of-code curve shown in Figure 1-2 against its own median and then performing the same normalization technique on the data in Figure 1-3 as well as data about particular incidents (also from CERT), Geer is able to overlay the three curves to begin to look for correlation (Figure 1-4). The curves fit best when the lines-of-code data are shifted right by two years, something that can be explained with reference to diffusion delay. This means that new operating system versions do not "plonk" into the world all at once in a massive coordinated switchover. Instead, there is a steady diffusion into the operating system population. A two-year diffusion delay seems logical.

Figure 1-4. Normalized versions of the millions of lines of code, vulnerabilities, and incidents data. Now that we have put these curves together, we can begin to compute curves for correlation and prediction. (After Geer.)

[View full size image]



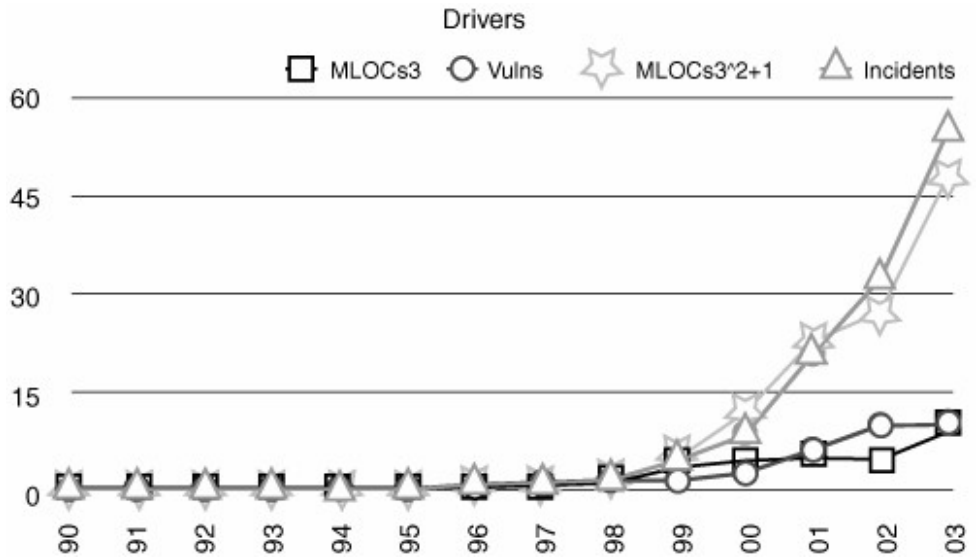
The next step is a bit more complex and involves some rolling average calculation. A code volume curve, which Geer calls MLOCs3 (millions of lines of code smoothed), is computed as the three-year moving average of code volume. A second such curve, called MLOCs3^2+1, is the square of the three-year moving average of code volume shifted right one year. Justification for the squaring operation comes from the commonly accepted rule of thumb that program complexity grows with the square of the number of lines of code. Given the resulting curves (shown in Figure 1-5), Geer argues:

Security faults are a subset of quality faults and the literature says that quality faults will tend to be a function of code complexity, itself proportional to the square of code volume. As such, the average complexity in the field should be a predictor of the attack-ability in an a priori sense. Shifting it right one year is to permit the attack community time to acquire access and skill to that growing code base complexity. This is not a statement of proven causality—it is exploratory data analysis.[10]

[10] Dan Geer, "The Physics of Digital Law," keynote address, CyberCrime and Digital Law Enforcement Conference, Information Society Project, Yale Law School, March 26, 2004. (Unpublished slides.)

Figure 1-5. Computation of two kinds of code volume curves (MLOCs3 and MLOCs3^2+1; see text for definition) results in curves with some predictive power. (After Geer.)

[View full size image]



Geer's analysis shows that intuitive claims about how growth in simple lines of code metrics correlates with growth in security problems actually hold analytical water.

To boil this all down to one line—more code, more bugs, more security problems

Security Problems in Software

Software security, that is, the process of designing, building, and testing software for security, gets to the heart of computer security by identifying and expunging problems in the software itself. In this way, software security attempts to build software that can withstand attack proactively.

Bugs and Flaws and Defects, Oh My!

Though Figure 1-1 clearly shows that the software problem is large, scientists have done little work in classifying and categorizing software security problems.

Perhaps the reintroduction of basic terminology—defect, bug, flaw, and risk—with a security emphasis can help clarify the categorization problem. I propose the following usage.

Defect: Both implementation vulnerabilities and design vulnerabilities are defects. A defect is a problem that may lie dormant in software for years only to surface in a fielded system with major consequences.

Bug: A bug is an implementation-level software problem. Bugs may exist in code but never be executed. Though the term bug is applied quite generally by many software practitioners, I reserve use of the term to encompass fairly simple implementation errors. Bugs are implementation-level problems that can be easily discovered and remedied. An

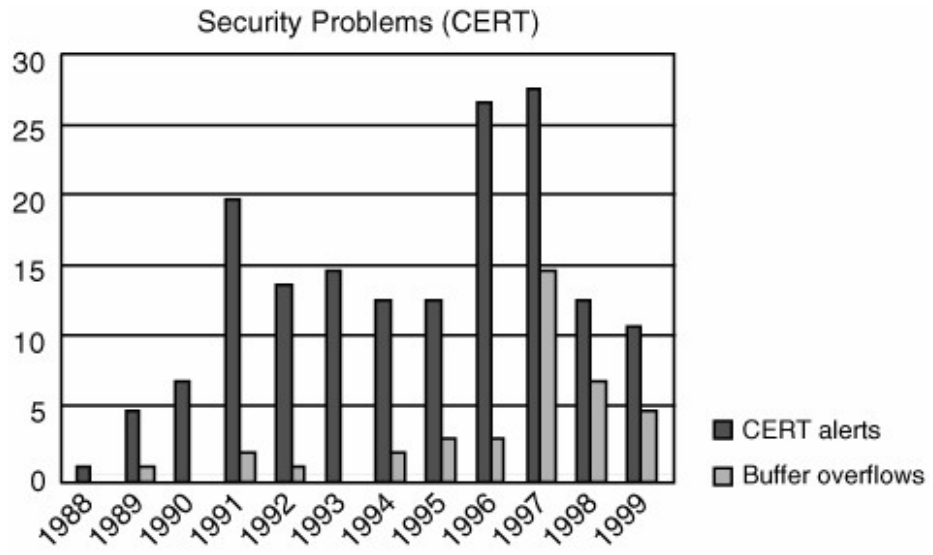
example of a bug is described in the following box—The (Much Ballyhoo'd) Buffer Overflow: An Implementation Bug.

Researchers have made significant progress in detecting security vulnerabilities stemming from low-level and mid-level implementation bugs. Early research tools include FIST [Ghosh, O'Connor, and McGraw 1998], ITS4 [Viega et al. 2000a], Jscan [Viega et al. 2000b], Splint [Evans et al. 1994], Metal [Engler et al. 2000], and Prefix/Prefast [Bush, Pincus, and Sielaff 2000]. Commercial tools (e.g., Fortify Software's Source Code Analyzer) were introduced to the market in 2005, and development of these tools continues apace. The tools are effective in detecting a wide range of implementation bugs, including buffer overflow vulnerabilities, format string bugs, resource leaks, and simple race conditions—all of which depend on only limited code analysis and knowledge of the external environment. (See Chapter 4 for more on code review and static analysis tool use.)

The (Much Ballyhoo'd) Buffer Overflow: An Implementation Bug

The most pervasive security problem today in terms of reported bugs is the buffer overflow. A now classic paper by Dave Wagner in 2000 looked over CERT data and determined that almost 45% of all software security problems reported to CERT were caused by buffer overflows. Figure 1-6 shows a copy of Wagner's data.

Figure 1-6. Dave Wagner's study determined the prevalence of buffer overflows as causes of CERT alerts (around 45%), showing how large a problem such buffer overflows are [Wagner et al. 2000].



The buffer overflow problem exists because it is so easy to write beyond the bounds of data objects in languages like C and C++. Type-safe languages like Java and C#, do not suffer from this issue, since the definition of exactly what constitutes a data object is much more tightly controlled.

In C, it is also extremely easy to allocate some bytes and then try to use more. The language does not care. For example, consider the two lines below:

```
char x[12];
x[12] = '\0';
```

In the code snippet, an array of 12 chars is declared. Then the thirteenth element is set to 0. Everyone in "the club" knows that for hazy historical reasons (offsets), array references in C start with 0! What a silly language.

There are two main flavors of buffer overflows: those associated with stack-allocated buffers and those associated with heap-allocated buffers. Overflowing a stack-allocated buffer is the most common attack. This is known as "smashing the stack." The C Programming Language (the C "bible") shows C programmers how they should never get input (without saying "never") [Kernighan and Ritchie 1988, p. 164]. Since we teach people to program in C as an introduction to programming, we should not be surprised at how common buffer overflow vulnerabilities are.

Many, many C library functions and arithmetic issues can lead to buffer overflows. Consider the snippet below. This is a dangerous piece of vulnerable code. Not only are we using `gets()` to get (unbounded) input, but we're using it to load a local variable on the stack. By providing just the right kind of input to this program, an attacker can obtain complete control over program control flow.

```
void main() {
  char buf[1024];
  gets(buf);
}
```

For more on buffer overflows, see Building Secure Software (where you are taught in excruciating detail how buffer overflows work) and Exploiting Software (which describes trampolining and other more advanced buffer overflow attacks, as well as plenty of real-world examples) [Viega and McGraw 2001; Hoglund and McGraw 2004].

If you are concerned about buffer overflow problems and other basic software

security bugs, don't use C. If you must use C, use a source code security scanner as described in Chapter 4.

By the way, C++ is even worse than C from a security perspective. C++ is C with an object model crammed halfway down its throat.

Flaw: A flaw is a problem at a deeper level. Flaws are often much more subtle than simply an off-by-one error in an array reference or use of an incorrect system call. A flaw is certainly instantiated in software code, but it is also present (or absent!) at the design level. For example, a number of classic flaws exist in error-handling and recovery systems that fail in an insecure or inefficient fashion. Another example can be found in the box, Microsoft Bob: A Design Flaw, that follows. Automated technologies to detect design-level flaws do not yet exist, though manual risk-analysis processes can identify flaws (see Chapter 5).

Table 1-2 provides some simple examples of bugs and flaws. In practice, we find that software security problems are divided 50/50 between bugs and flaws. This means that eradicating bugs through code review will solve only about half of the problem. This may come as a big surprise to those people who believe that software security is exclusively about coding issues. Clearly, it isn't. Microsoft reports that more than 50% of the problems the company has uncovered during its ongoing security push are architectural in nature [Mike Howard, personal communication]. Cigital data show a 60/40 split in favor of flaws, reflecting Cigital's specialization in architectural risk analysis.

Table 1-2. Examples of Bugs and Flaws

Bugs	**Flaws**
Buffer overflow: stack smashing	Method over-riding problems (subclass issues)
Buffer overflow: one-stage attacks	Compartmentalization problems in design
Buffer overflow: string format attacks	Privileged block protection failure (`DoPrivilege()`)
Race conditions: TOCTOU	Error-handling problems (fails open)
Unsafe environment variables	Type safety confusion error
Unsafe system calls (`fork()`, `exec()`, `system()`)	Insecure audit log design
	Broken or illogical access control (role-based access control [RBAC] over tiers)

Table 1-2. Examples of Bugs and Flaws

Bugs	Flaws
Incorrect input validation (black list vs. white list)	Signing too much code
Software security defects come in two basic flavors, each of which accounts for approximately 50% of software security problems.	

Microsoft Bob: A Design Flaw

This is an oft-repeated story that may be apocryphal, but it is amusing and teaches an interesting lesson.

Microsoft's Bob program was meant as a helper for Windows ME and Windows 98. Though the security posture of these early PC operating systems is known to be very poor, Windows ME did include a facility for setting a system password.

Microsoft Bob would pipe up (like Clippie the Paperclip in Word) when the program determined that the user was stuck doing something. Bob's most insecure function occurred when a user attempted three times (unsuccessfully) to type in his or her password. Bob would pop up and proclaim: "I see you have forgotten your password, please enter a new password." Then the user was allowed to change the password even though the user apparently had no idea of the old one.

Microsoft Bob, hacker's friend.

Risk: Flaws and bugs lead to risk. Risks are not failures. Risks capture the probability that a flaw or a bug will impact the purpose of the software (that is, risk = probability x impact). Risk measures must also take into account the potential damage that can occur. A very high risk is not only likely to happen but also likely to cause great harm. Risks can be managed by technical and non-technical means.

Building secure software is like building a house. I liken correct low-level coding (such as using functions likely to cause buffer overflows) to the use of solid bricks as opposed to bricks made of sawdust. The kinds of bricks used are important to the integrity of the house, but even more important (if the goal is to keep bad things out) is having four walls and a roof in the design. The same thing goes for software: Which system calls and libraries are used and how they are used is important, but overall design properties often count for more. In general, software security to date has paid much more attention to bricks than to walls.

The Range of Defects

Drawing a hard-and-fast distinction between bugs and flaws is nice, but in practice things are much messier. Sometimes determining whether a defect is a flaw or a bug is difficult. That's because flaws and bugs exist along a continuum of defects. Security defects in software systems range from local implementation errors (e.g., use of the `gets()` function call in C/C++) to interprocedural interface errors (e.g., a race condition between an access control check and a file operation) to much higher design-level mistakes (e.g., error-handling and recovery systems that fail in an insecure fashion or object-sharing systems that mistakenly include transitive trust issues).

We can consider these defects as defining a large range based on how much program code must be considered to understand the vulnerability, how much detail regarding the execution environment must be known to understand the vulnerability, and whether a design-level description is best for determining whether or not a given vulnerability is present. For example, we can determine that a call to `gets()` in a C/C++ program can be exploited in a buffer overflow attack without knowing anything about the rest of the code, its design, or the execution environment other than assuming that the user entering text on standard input may be malicious. Hence, a `gets()` vulnerability can be detected with good precision using a very simple lexical analysis. This kind of approach is the subject of Chapter 4. A taxonomy of low-level coding defects can be found in Chapter 12.

Midrange vulnerabilities involve interactions among more than one location in code. Precisely detecting race conditions, for example, depends on more than simply analyzing an isolated line of code—it may depend on knowing about the behavior of several functions, understanding sharing among global variables, and being familiar with the operating system providing the execution environment.

Design-level vulnerabilities carry this trend further. Unfortunately, ascertaining whether or not a program has design-level vulnerabilities requires great expertise (and is the subject of Chapter 5). This makes finding design-level flaws not only hard to do but particularly hard to automate as well. The problem is that design-level problems appear to be prevalent and are at the very least a critical category of security risk in code.

Consider an error-handling and recovery system. Failure recovery is an essential aspect of security engineering. But it is complicated because it interacts with failure models, redundant design, and defense against denial-of-service attacks. Understanding whether or not an error-handling and recovery system in an object-oriented program is secure, for example, involves figuring out a global property spread throughout many classes in typical design. Error detection code is usually present in each object and method, and error-handling code is usually separate and distinct from the detection code. Sometimes exceptions propagate up to the system level and are handled by the machine running the code (e.g., Java 2 Virtual Machine exception handling). This makes determining whether or not a given error-handling and recovery design is secure quite difficult. The problem is ex-acerbated in transaction-based systems commonly used in commercial e-commerce

solutions where functionality is distributed among many different components running on several servers.

Other examples of design-level problems include object-sharing and trust issues, unprotected data channels (both internal and external), incorrect or missing access control mechanisms, lack of auditing/logging or incorrect logging, ordering and timing errors (especially in multithreaded systems), and many others. In order to make progress as a scientific discipline, software security professionals must understand and categorize these sorts of problems in a rigorous way.

The Problem with Application Security

Because the idea that software is a major problem in computer security is fairly new, many diverse sets of people are working on the problem. One set of network security practitioners, led by a number of security tools vendors, has worked hard and spent lots of marketing money to coin "application security" as the moniker of choice to describe the software security space. There are a number of reasons to be wary when confronted with application security. Personally, I am a proponent of the term software security over the term application security, especially when discussing the idea of building security in. Here's why.

One problem is that the term application security means different things to different people. In many circles, it has come to mean the protection of software after it's already built. Although the notion of protecting software is an important one, it's just plain easier to protect something that is defect-free than something riddled with vulnerabilities.

Pondering the question, "What is the most effective way to protect software?" can help untangle software security and application security. On one hand, software security is about building secure software: designing software to be secure; making sure that software is secure; and educating software developers, architects, and users about how to build security in. On the other hand, application security is about protecting software and the systems that software runs in a post facto way, only after development is complete. Issues critical to this subfield include sandboxing code (as the Java Virtual Machine does), protecting against malicious code, obfuscating code, locking down executables, monitoring programs as they run (especially their input), enforcing the software-use policy with technology, and dealing with extensible systems.

Application security follows naturally from a network-centric approach to security by embracing standard approaches, such as "penetrate and patch" and input filtering (trying to block malicious input), and by generally providing value in a reactive way. (See the next box—Application Security Testing Tools: Good or Bad?) Put succinctly, application security is based primarily on finding and fixing known security problems after they've been exploited in fielded systems, usually by filtering dangerous input on its way to broken software. Software security—the process of designing, building, and testing software for security—identifies and expunges problems in the software itself. In this way, software security practitioners attempt to build software that can withstand attack

proactively. Let me give you a specific example: Although there is some real value in stopping buffer overflow attacks by observing HTTP traffic as it arrives over port 80, a superior approach is to fix the broken code in order to avoid the buffer overflow completely.

Another problem I have with the term application security is that it unnecessarily limits the purview of software security. Sure, applications have security problems, with Web-based applications leading the pack. But if you step back a moment, you'll see that we have a much bigger problem at hand than simply errant Web applications. Ask yourself, what do wireless devices, cell phones, PDAs, browsers, operating systems, routers, servers, personal computers, public key infrastructure systems, and firewalls have in common? The answer is "software." What an interesting and wide-ranging list. It encompasses everything from consumer devices to infrastructure items to security apparatus itself. We should not be surprised that real attackers go after bad software—no matter where it lives. A myopic focus on "application" code ignores the bigger picture. That's why I like to call the field software security.

It is important to think about the impact of simple vocabulary choices in large enterprises. When a large organization sets an application development project in motion, it involves lots of diverse groups: systems people, network people, the architecture group, and a whole bevy of application developers. If the security group buys into application security thinking, they'll likely end up pushing some vendor or product at their applications people (the VB.NET implementers at the bottom of the software food chain). By contrast, software security thinking focuses its scrutiny on both the applications people and those middleware architects responsible for all of the hard-core "services" code that is extremely susceptible to design flaws. (Of course, both application code and the middleware services it relies on can possess bugs.)

Suborganizations like application development and the architecture group are very territorial, and even if the vendor or product chosen as an application security solution does end up finding defects in the application, the people in the cross hairs are likely to pass the buck: "Oh, you need to talk to the architects." The security ball has a big chance of being dropped in this situation—especially since the architecture and "real" code is usually set in stone and the architects redispatched to other projects before the VB.NET application implementers are even contracted.

Application Security Testing Tools: Good or Bad?[*]

Application security testing products are being sold as a solution to the problem of insecure software. Unfortunately, these first-generation solutions are not all they are cracked up to be. They may help us diagnose, describe, and demonstrate the problem, but they do little to help us fix it.

Today's application security products treat software applications as "black

boxes" that are prone to misbehave and must be probed and prodded to prevent security disaster. Unfortunately, this approach is too simple.

Software testing requires planning and should be based on software requirements and the architecture of the code under test. You can't "test quality in" by painstakingly finding and removing bugs once the code is done. The same goes for security; running a handful of canned tests that "simulate malicious hackers" by sending malformed input streams to a program will not work. Real attackers don't simply "fuzz" a program with input to find problems. Attackers take software apart, determine how it works, and make it misbehave by doing what users are not supposed to do. The essence of the disconnect is that black box testing approaches, including application security testing tools, only scratch the surface of software in an outside⟶in fashion instead of digging into the guts of software and securing things from the inside.

Badness-ometers

That said, application security testing tools can tell you something about security—namely, that you're in very deep trouble. That is, if your software fails any of the canned tests, you have some serious security work to do. The tools can help uncover known issues. But if you pass all the tests with flying colors, you know nothing more than that you passed a handful of tests with flying colors.

Put in more basic terms, application security testing tools are "badness-ometers," as shown in Figure 1-7. They provide a reading in a range from "deep trouble" to "who knows," but they do not provide a reading into the "security" range at all. Most vulnerabilities that exist in the architecture and the code are beyond the reach of simple canned tests, so passing all the tests is not that reassuring. (Of course, knowing you're in deep trouble can be helpful!)

Figure 1-7. A badness-ometer can be useful in some cases but is not the same thing as a security-ometer.



The other major weakness with application security testing tools is that they focus only on input to an application provided over port 80. Understanding and testing a complex program by relying only on the protocol it uses to communicate provides a shallow analysis. Though many attacks do arrive via HTTP, this is only one category of security problem. First of all, input arrives to modern applications in many forms other than HTTP: consider SSL, environment variables, outside libraries, distributed components that communicate using other protocols, and so on. Beyond program input, software security must consider architectural soundness, data security, access control, software environment, and any number of other aspects, all of which are dependent on the application itself. There is no set of prefab tests that will probe every possible application in a meaningful way.

The only good use for application security tools is testing commercial off-the-shelf software. Simple dynamic checks set a reasonably low bar to hold vendors to. If software that is delivered to you fails to pass simple tests, you can either reject it out of hand or take steps to monitor its behavior.

In the final analysis, application security testing tools do provide a modicum of value. Organizations that are just beginning to think through software security issues can use them as badness-ometers to help determine how much trouble they are in. Results can alert all the interested parties to the presence of the problem and motivate some mitigation activity. However, you won't get anything more than a rudimentary analysis with these tools. Fixing the problems they expose requires building better software to begin with—whether you created the software or not.

[*] A version of this example first appeared in my "[In]security" column in Network magazine, November 2004. Network magazine is now called IT Architect.

Software Security and Operations

One reason that application security technologies, such as application firewalls, have evolved the way they have is because operations people dreamed them up. In most corporations and large organizations, security is the domain of the infrastructure people who set up and maintain firewalls, intrusion detection systems, and antivirus engines (all of which are reactive technologies).

However, these people are operators, not builders. Given the fact that they don't build the software they have to operate, it's no surprise that their approach is to move standard security techniques "down" to the desktop and application levels. The gist of the idea is to protect vulnerable things (in this case, software) from attack, but the problem is that vulnerabilities in the software let malicious hackers skirt standard security technologies with impunity. If this were not the case, the security vulnerability problem would not be expanding the way it is. Clearly, this emphasizes the need to get builders to do a better job on the software in the first place. (See the Security versus Software box.)

Protecting a network full of evolving software is difficult, even if the software is not patched every five minutes. If software were in some sense self-protecting (by being designed defensively and more properly tested from a security perspective) or at least less riddled with vulnerabilities, running a secure network could become easier and more cost effective.

In the short run, we clearly—desperately—must make progress on both fronts. But in the long run, we must figure out ways to build easier-to-defend code. Software security is about helping builders do a better job so that operators end up with an easier job.

Security versus Software

Security Has Come a Long Way

Security was the exclusive domain of guns, dogs, and concrete not too many years ago. Since the worldwide deluge that is the Information Age, all things security have changed radically. In tandem with the stunning growth of the Internet, the new field of computer security has taken root and grown like a weed. Computer security quickly became everyone's business as commerce, entertainment, and personal communications were swept up in the Internet flood. Yet computer security remains a relative newcomer.

In the early days, computer security was about protecting the expensive machine from people (remember when computers took up entire rooms?). There were no networks, and there were not really that many users. Operations people ruled the roost.

Once things shrank to a more reasonable size and the network was invented,

computer security confronted its first major shift. The trusted machine was connected to untrusted machines not necessarily under the control of operations people. And the dang things could be anywhere. The need for network security was paramount, so a host of reactive technologies came into being, including the firewall, antivirus programs, and intrusion detection systems. Computer security on the Internet relies on these technologies to this day. Operations people continue to rule the roost.

The problem is that, though certainly necessary, the kinds of common computer security technologies we are counting on today simply don't work well enough. Take a look at any study, from the annual CSI/FBI report to CERT findings to reports commissioned by NIST—by every measure the computer security problem is growing even though adoption of network security technologies continues unabated. Why?

Security Has Not Come Very Far

Defending any human artifact against malicious adversaries is difficult. This is a lesson from way back in the days of physical security. The notion of "defending the perimeter," adapted from the physical security of castles and fortresses, requires the existence of a perimeter. Some castles and fortresses were better designed than others, and as a result they were easier to defend.

The perimeter defense paradigm has its issues, though. Consider the Maginot Line, built as a perimeter defense in France against German aggression after World War I. The problem was that the defense failed when the attackers changed their traditional invasion routes and came through Ardennes Forest and once-neutral Belgium.

Computer security has come to rely too heavily on a perimeter defense mentality, and the attackers have already changed their invasion routes. The perimeter metaphor makes sense if you take the view that the trusted inside machines need to be protected from the untrusted machines outside. The problem is that the notion of a perimeter is quaint, outdated, and too simple to work. Today's Web-based systems are highly distributed and involve explicit connection with machines that merit varying degrees of trust. Reactive technologies, such as firewalls that attempt to protect "the system" from the "outside," don't work when the very design of the system involves tunneling through the firewall with impunity.

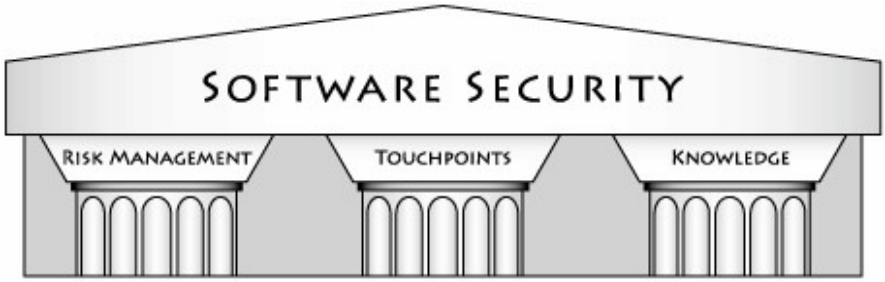
Solving the Problem: The Three Pillars of Software Security

Software security is an ongoing activity that requires a cultural shift. There is unfortunately no magic tool or just-add-water process that will result in secure software. Software security takes work. That's the bad news. The good news is that any

organization that is developing software, no matter what software development methodology it is following (if any!), can make straightforward, positive progress by following the plan laid out in this book.

Software security naturally borrows heavily from software engineering, programming languages, and security engineering. The three pillars of software security are applied risk management, software security touchpoints, and knowledge (see Figure 1-8). By applying the three pillars in a gradual, evolutionary manner and in equal measure, a reasonable, cost-effective software security program can result. Throughout the rest of this book, I discuss the three pillars and their constituent parts at length.

Figure 1-8. The three pillars of software security are risk management, software security touchpoints, and knowledge.





Pillar I: Applied Risk Management

No discussion about security is complete without considering risk management, and the same holds true for software security. To make risk management coherent, it is useful to draw a distinction between the application of risk analysis at the architectural level (sometimes called threat modeling or security design analysis) and the notion of tracking and mitigating risk as a full lifecycle activity. Architectural risk analysis is a best practice and is one of the central touchpoints (see Chapter 5). However, security risks crop up throughout the software development lifecycle (SDLC); thus, an overall approach to risk management as a philosophy is also important. I will call this underlying approach the risk management framework (RMF).

Risk management is often viewed as a "black art"—that is, part fortune-telling, part mathematics. Successful risk management, however, is nothing more than a business-level decision-support tool: a way to gather the requisite data to make a good judgment call, based on knowledge of vulnerabilities, threats, impacts, and probabilities. Risk management has a storied history. Dan Geer wrote an excellent overview [Geer 1998]. What makes a good software risk assessment is the ability to apply classic risk definitions to software designs in order to generate accurate mitigation requirements.

Chapter 2 discusses an RMF and considers applied risk management as a high-level approach to iterative risk identification and mitigation that is deeply integrated throughout the SDLC. Carrying out a full lifecycle risk management approach for software security is at its heart a philosophy underpinning all software security work. The basic idea is to identify, rank, track, and understand software security risk as the touchpoints are applied throughout the SDLC.

Chapter 5 provides a discussion of architectural risk analysis. In that chapter I briefly introduce some practical methods for applying risk analysis techniques while software is being designed and built. There are many different, established methodologies, each possessing distinct advantages and disadvantages.



Pillar II: Software Security Touchpoints

On the road to implementing a fundamental change in the way we build software, we must first agree that software security is not security software. This is a subtle point often lost on development people, who tend to focus on functionality. Obviously, there are security functions in the world, and most modern software includes security features; but adding features such as SSL to your program (to cryptographically protect communications) does not present a complete solution to the security problem. Software security is a system-wide issue that takes into account both security mechanisms (such as access control) and design for security (such as robust design that makes software attacks difficult). Sometimes these overlap, but often they don't.

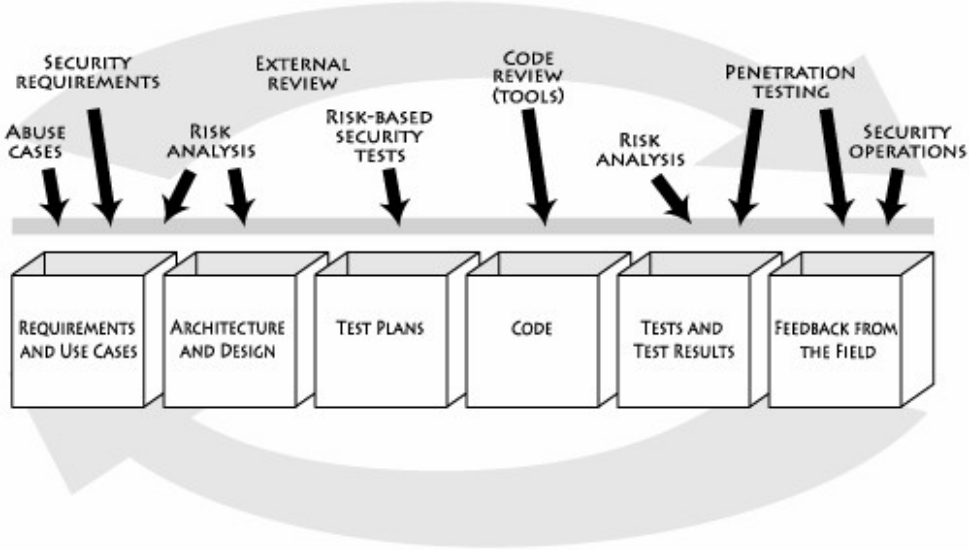
Put another way, security is an emergent property of a software system. A security problem is more likely to arise because of a problem in a system's standard-issue part (say, the interface to the database module) than in some given security feature. This is an important reason why software security must be part of a full lifecycle approach. Just as you can't test quality into a piece of software, you can't spray paint security features onto a design and expect it to become secure. There's no such thing as magic crypto fairy dust—we need to focus on software security from the ground up. We need to build security in.

As practitioners become aware of software security's importance, they are increasingly adopting and evolving a set of best practices to address the problem. Microsoft has carried out a noteworthy effort under its Trustworthy Computing Initiative [Walsh 2003; Howard and Lipner 2003]. (See the next box, Microsoft's Trustworthy Computing Initiative.) Most approaches in practice today encompass training for developers, testers, and architects; analysis and auditing of software artifacts; and security engineering. In the fight for better software, treating the disease itself (poorly designed and implemented software) is better than taking an aspirin to stop the symptoms. There's no substitute for working software security as deeply into the development process as possible and taking advantage of the engineering lessons software practitioners have learned over the years.

Figure 1-9 specifies the software security touchpoints (a set of best practices) that I cover in this book and shows how software practitioners can apply the touchpoints to the various software artifacts produced during software development. These best practices first appeared as a set in 2004 in IEEE Security & Privacy magazine [McGraw 2004]. Since then, they have been adopted (and in some cases adapted) by the U.S. government in the National Cyber Security Task Force report [Davis et al. 2004], by Cigital, by the U.S. Department of Homeland Security, and by Ernst and Young. In various chapters ahead, I'll detail these best practices (see Part II).

Figure 1-9. Software security best practices applied to various software artifacts. Although in this picture the artifacts are laid out according to a traditional waterfall model, most organizations follow an iterative approach today, which means that best practices will be cycled through more than once as the software evolves.

[View full size image]



Microsoft's Trustworthy Computing Initiative

The Gates memo of January 2002 reproduced here highlights the importance of building secure software to the future of Microsoft. Microsoft's Trustworthy Computing Initiative, kicked off by the memo, has changed the way Microsoft builds software. Microsoft has spent more than $300 million (and more than 2000 worker days) on its software security push.

Microsoft is focusing on people, process, and technology to tackle the software security problem. On the people front, Microsoft is training every developer, tester, and program manager in basic techniques of building secure products. Microsoft's development process has been enhanced to make security a critical factor in design, coding, and testing of every product. Risk analysis, code

review, and security testing all have their place in the new process. External review and testing also play a key role. Microsoft is pursuing software security technology by building tools to automate as many process steps as possible. Tools include Prefix and Prefast for defect detection [Bush, Pincus, and Sielaff 2000] and changes to the Visual C++ compiler to detect certain kinds of buffer overruns at runtime. Microsoft has also recently begun thinking about measurement and metrics for security.

Microsoft has experimented with different ways to integrate software security practices into the development lifecycle. The company's initial approach is shown in Figure 1-10. This picture, originally created by Mike Howard, helped to inspire the process-agnostic touchpoints approach described in this book. Howard's original approach is very much Microsoft-centric (in that it is tied to the Microsoft product lifecycle and is not process agnostic), but it does emphasize the importance of a full-lifecycle approach.

Figure 1-10. Early on, Microsoft put into place the (Microsoft-centric) software security process shown here. Notice that security does not happen at one lifecycle stage; nor are constituent activities "fire and forget."

© 2005 Microsoft Corporation. All rights reserved. Reprinted with permission.

[View full size image]

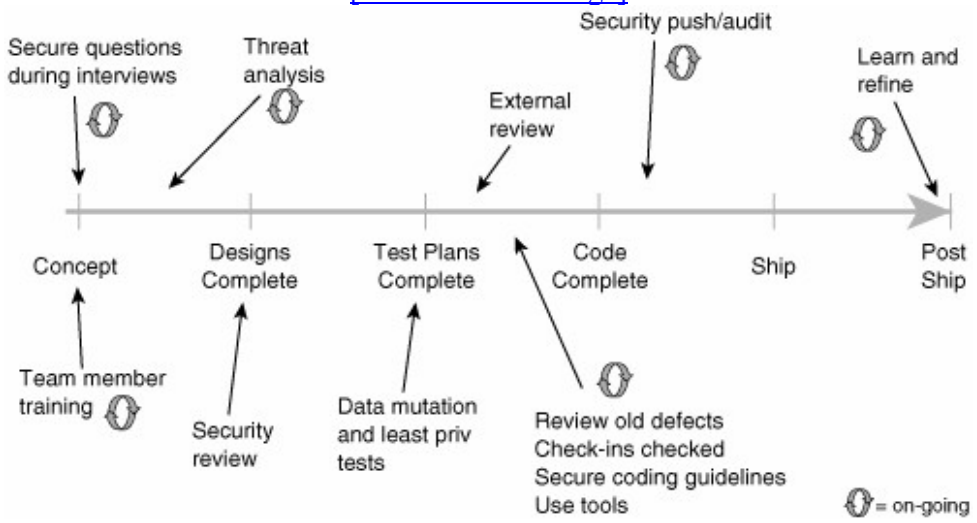
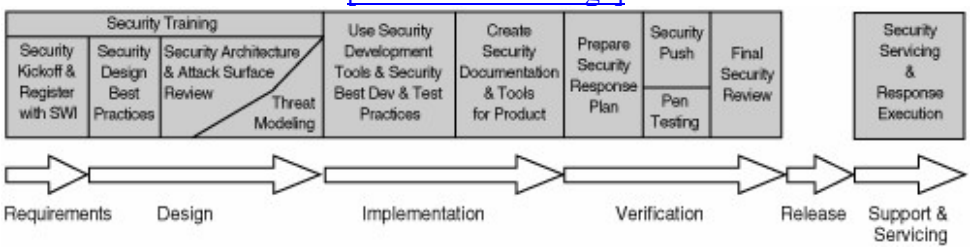


Figure 1-11 shows a more up-to-date version of Microsoft's process for software security. A detailed paper describing the current version of Microsoft's Trustworthy Computing Secure Development Lifecycle is available on the Web through MSDN.[*]

Figure 1-11. An updated view of Microsoft's software security process. [*]

© 2005 Microsoft Corporation. All rights reserved. Reprinted with permission.

[View full size image]



The Gates Memo[†]

The refocusing of Microsoft to pay more attention to security was sparked by Bill Gates himself. In an e-mail sent to all Microsoft employees in January 2002 and widely distributed on the Internet (see <http://news.com.com/2009-1001-817210.html>), Microsoft Chairman Bill Gates started a major shift at Microsoft away from a focus on features to building more secure and trustworthy software. The e-mail is reproduced in its entirety here.

From: Bill Gates

Sent: Tuesday, January 15, 2002 2:22 PM

To: Microsoft and Subsidiaries: All FTE

Subject: Trustworthy computing

Every few years I have sent out a memo talking about the highest priority for Microsoft. Two years ago, it was the kickoff of our .NET strategy. Before that, it was several memos about the importance of the Internet to our future and the ways we could make the Internet truly useful for people. Over the last year it has become clear that ensuring .NET as a platform for Trustworthy Computing is more important than any other part of our work. If we don't do this, people simply won't be willing—or able—to take advantage of all the other great work we do. Trustworthy Computing is the highest priority for all the work we are doing. We must lead the industry to a whole new level of Trustworthiness in computing.

When we started work on Microsoft .NET more than two years ago, we set a new direction for the company—and articulated a new way to think about our software. Rather than developing standalone applications and Web sites, today we're moving towards smart clients with rich user interfaces interacting with

Web services. We're driving the XML Web services standards so that systems from all vendors can share information, while working to make Windows the best client and server for this new era.

There is a lot of excitement about what this architecture makes possible. It allows the dreams about e-business that have been hyped over the last few years to become a reality. It enables people to collaborate in new ways, including how they read, communicate, share annotations, analyze information and meet.

However, even more important than any of these new capabilities is the fact that it is designed from the ground up to deliver Trustworthy Computing. What I mean by this is that customers will always be able to rely on these systems to be available and to secure their information. Trustworthy Computing is computing that is as available, reliable and secure as electricity, water services and telephony.

Today, in the developed world, we do not worry about electricity and water services being available. With telephony, we rely both on its availability and its security for conducting highly confidential business transactions without worrying that information about who we call or what we say will be compromised. Computing falls well short of this, ranging from the individual user who isn't willing to add a new application because it might destabilize their system, to a corporation that moves slowly to embrace e-business because today's platforms don't make the grade.

The events of last year—from September's terrorist attacks to a number of malicious and highly publicized computer viruses—reminded every one of us how important it is to ensure the integrity and security of our critical infrastructure, whether it's the airlines or computer systems.

Computing is already an important part of many people's lives. Within ten years, it will be an integral and indispensable part of almost everything we do. Microsoft and the computer industry will only succeed in that world if CIOs, consumers and everyone else see that Microsoft has created a platform for Trustworthy Computing.

Every week there are reports of newly discovered security problems in all kinds of software, from individual applications and services to Windows, Linux, Unix and other platforms. We have done a great job of having teams work around the clock to deliver security fixes for any problems that arise. Our responsiveness has been unmatched—but as an industry leader we can and must do better. Our new design approaches need to dramatically reduce the number of such issues that come up in the software that Microsoft, its partners and its customers create. We need to make it automatic for customers to get the benefits of these fixes. Eventually, our software should be so fundamentally secure that customers

never even worry about it.

No Trustworthy Computing platform exists today. It is only in the context of the basic redesign we have done around .NET that we can achieve this. The key design decisions we made around .NET include the advances we need to deliver on this vision. Visual Studio .NET is the first multi-language tool that is optimized for the creation of secure code, so it is a key foundation element.

I've spent the past few months working with Craig Mundie's group and others across the company to define what achieving Trustworthy Computing will entail, and to focus our efforts on building trust into every one of our products and services. Key aspects include:

Availability: Our products should always be available when our customers need them. System outages should become a thing of the past because of a software architecture that supports redundancy and automatic recovery. Self-management should allow for service resumption without user intervention in almost every case.

Security: The data our software and services store on behalf of our customers should be protected from harm and used or modified only in appropriate ways. Security models should be easy for developers to understand and build into their applications.

Privacy: Users should be in control of how their data is used. Policies for information use should be clear to the user. Users should be in control of when and if they receive information to make best use of their time. It should be easy for users to specify appropriate use of their information including controlling the use of email they send.

Trustworthiness is a much broader concept than security, and winning our customers' trust involves more than just fixing bugs and achieving "five-nines" availability. It's a fundamental challenge that spans the entire computing ecosystem, from individual chips all the way to global Internet services. It's about smart software, services and industry-wide cooperation.

There are many changes Microsoft needs to make as a company to ensure and keep our customers' trust at every level—from the way we develop software, to our support efforts, to our operational and business practices. As software has become ever more complex, interdependent and interconnected, our reputation as a company has in turn become more vulnerable. Flaws in a single Microsoft product, service or policy not only affect the quality of our platform and services overall, but also our customers' view of us as a company.

In recent months, we've stepped up programs and services that help us create better software and increase security for our customers. Last fall, we launched

the Strategic Technology Protection Program, making software like IIS and Windows .NET Server secure by default, and educating our customers on how to get—and stay—secure. The error-reporting features built into Office XP and Windows XP are giving us a clear view of how to raise the level of reliability. The Office team is focused on training and processes that will anticipate and prevent security problems. In December, the Visual Studio .NET team conducted a comprehensive review of every aspect of their product for potential security issues. We will be conducting similarly intensive reviews in the Windows division and throughout the company in the coming months.

At the same time, we're in the process of training all our developers in the latest secure coding techniques. We've also published books like Writing Secure Code, by Michael Howard and David LeBlanc, which gives all developers the tools they need to build secure software from the ground up. In addition, we must have even more highly trained sales, service and support people, along with offerings such as security assessments and broad security solutions. I encourage everyone at Microsoft to look at what we've done so far and think about how they can contribute.

But we need to go much further.

In the past, we've made our software and services more compelling for users by adding new features and functionality, and by making our platform richly extensible. We've done a terrific job at that, but all those great features won't matter unless customers trust our software. So now, when we face a choice between adding features and resolving security issues, we need to choose security. Our products should emphasize security right out of the box, and we must constantly refine and improve that security as threats evolve. A good example of this is the changes we made in Outlook to avoid email borne viruses. If we discover a risk that a feature could compromise someone's privacy, that problem gets solved first. If there is any way we can better protect important data and minimize downtime, we should focus on this. These principles should apply at every stage of the development cycle of every kind of software we create, from operating systems and desktop applications to global Web services.

Going forward, we must develop technologies and policies that help businesses better manage ever larger networks of PCs, servers and other intelligent devices, knowing that their critical business systems are safe from harm. Systems will have to become self-managing and inherently resilient. We need to prepare now for the kind of software that will make this happen, and we must be the kind of company that people can rely on to deliver it.

This priority touches on all the software work we do. By delivering on Trustworthy Computing, customers will get dramatically more value out of our advances than they have in the past. The challenge here is one that Microsoft is

uniquely suited to solve.

Bill

[*] Steve Lipner and Michael Howard "The Trustworthy Computing Security Development Lifecycle," MSDN, March 2005, Security Engineering and Communications, Security Business and Technology Unit, Microsoft Corporation <http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dnsecure/html/sdl.asp>.

[†] The complete Gates memo is included with permission from Microsoft.

Note that software security touchpoints can be applied regardless of the base software process being followed. Software development processes as diverse as the waterfall model, Rational Unified Process (RUP), eXtreme Programming (XP), Agile, spiral development, Capability Maturity Model integration (CMMi), and any number of other processes involve the creation of a common set of software artifacts (the most common artifact being code). In the end, this means you can create your own Secure Development Lifecycle (SDL) by adapting your existing SDLC to include the touchpoints. You already know how to build software; what you may need to learn is how to build secure software.

The artifacts I will focus on (and describe best practices for) include requirements and use cases, architecture, design documents, test plans, code, test results, and feedback from the field. Most software processes describe the creation of these kinds of artifacts. In order to avoid the "religious warfare" surrounding which particular software development process is best, I introduce this notion of artifact and artifact analysis. The basic idea is to describe a number of microprocesses (touchpoints or best practices) that can be applied inline regardless of your core software process.[11]

[11] Worth noting is the fact that I am not a process wonk by any stretch of the imagination. If you don't believe me, check out Chapter 1 of my software engineering book Software Fault Injection [Voas and McGraw 1998].

This process-agnostic approach to the problem makes the software security material explained in this book as easy as possible to adopt. This is particularly critical given the fractional state of software process adoption in the world. Requiring that an organization give up, say, XP and adopt RUP in order to think about software security is ludicrous. The good news is that my move toward process agnosticism seems to work out. I consider the problem of how to adopt these best practices for any particular software methodology beyond the scope of this book (but work that definitely needs to be done).



Pillar III: Knowledge

One of the critical challenges facing software security is the dearth of experienced practitioners. Early approaches that rely solely on apprenticeship as a method of propagation will not scale quickly enough to address the burgeoning problem. As the field evolves and best practices are established, knowledge management and training play a central role in encapsulating and spreading the emerging discipline more efficiently. Pillar III involves gathering, encapsulating, and sharing security knowledge that can be used to provide a solid foundation for software security practices.

Knowledge is more than simply a list of things we know or a collection of facts. Information and knowledge aren't the same thing, and it is important to understand the difference. Knowledge is information in context—information put to work using processes and procedures. A checklist of potential security bugs in C and C++ is information; the same information built into a static analysis tool is knowledge.

Software security knowledge can be organized into seven knowledge catalogs (principles, guidelines, rules, vulnerabilities, exploits, attack patterns, and historical risks) that are in turn grouped into three knowledge categories (prescriptive knowledge, diagnostic knowledge, and historical knowledge).

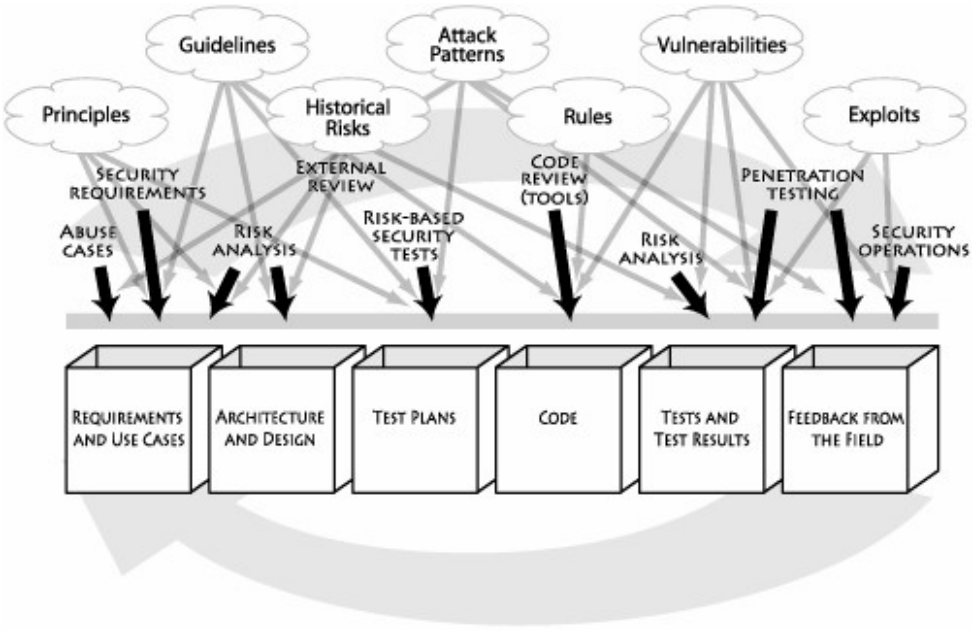
Two of these seven catalogs—vulnerabilities and exploits—are likely to be familiar to software developers possessing only a passing familiarity with software security. These catalogs have been in common use for quite some time and have even resulted in collection and cataloging efforts serving the security community. Similarly, principles (stemming from the seminal work of Saltzer and Schroeder [1975]) and rules (identified and captured in static analysis tools such as ITS4 [Viega et al. 2000a]) are fairly well understood. Knowledge catalogs only more recently identified include guidelines (often built into prescriptive frameworks for technologies such as .NET and J2EE), attack patterns [Hoglund and McGraw 2004], and historical risks. Together, these various knowledge catalogs provide a basic foundation for a unified knowledge architecture supporting software security.

Software security knowledge can be successfully applied at various stages throughout the entire SDLC. One effective way to apply such knowledge is through the use of software security touchpoints. For example, rules are extremely useful for static analysis and code review activities.

Figure 1-12 shows an enhanced version of the software security touchpoints diagram introduced in Figure 1-9. In Figure 1-12, I identify those activities and artifacts most clearly impacted by the knowledge catalogs briefly mentioned above. More information about these catalogs can be found in Chapter 11.

Figure 1-12. Mapping of software security knowledge catalogs to various software artifacts and software security best practices.

[View full size image]



Awareness of the software security problem is growing among researchers and some security practitioners. However, the most important audience has in some sense experienced the least exposure—for the most part, software architects, developers, and testers remain blithely unaware of the problem. One obvious way to spread software security knowledge is to train software development staff on critical software security issues.

The most effective form of training begins with a description of the problem and demonstrates its impact and importance. During the Windows security push in February and March 2002, Microsoft provided basic awareness training to all of its developers. Many other organizations have ongoing software security awareness training programs. Beyond awareness, more advanced software security training should offer coverage of security engineering, design principles and guidelines, implementation risks, design flaws, analysis techniques, and security testing. Special tracks should be made available to quality assurance personnel, especially those who carry out testing.

Of course, the best training programs will offer extensive and detailed coverage of the touchpoints covered in this book. Putting the touchpoints into practice requires cultural change, and that means training. Assembling a complete software security program at the enterprise level is the subject of Chapter 10.

The good news is that the three pillars of software security—risk management, touchpoints, and knowledge—can be applied in a sensible, evolutionary manner no matter what your existing software development approach is

The Rise of Security Engineering

Designers of modern systems must take security into account proactively. This is especially true when it comes to software because bad software lies at the heart of a majority of computer security problems. Software defects come in two flavors—design-level flaws and implementation bugs. To address both kinds of defects, we must build better software and design more secure systems from the ground up.

Most computer security practitioners today are operations people. They are adept at designing reasonable network architectures, provisioning firewalls, and keeping networks up. Unfortunately, many operations people have only the most rudimentary understanding of software. This leads to the adoption of weak reactive technologies (think "application security testing" tools). Tools like those target the right problem (software) with the wrong solution (outside⟶in testing).

Fortunately, things are beginning to change in security. Practitioners understand that software security is something we need to work hard on. The notion that it is much cheaper to prevent than to repair helps to justify investment up front. In the end, prevention technology and assurance best practices may be the only way to go. Microsoft's Trustworthy Computing Initiative is no accident.

If we are to build systems that can be properly operated, we must involve the builders of systems in security. This starts with education, where security remains an often-unmentioned specialty, especially in the software arena. Every modern security department needs to think seriously about security engineering. The best departments already have staff devoted to software security. Others are beginning to look at the problem of security engineering. At the very least, close collaboration with the "builders" in your organization is a necessity.

Don't forget that software security is not just about building security functionality and integrating security features! Coders are likely to ask, "If I use [this API], is it good enough?" when doing their building thing. The question to ask in response is, "What attacks would have serious impact and are worth avoiding for this module?" This line of questioning works to elicit a better understanding of design and its security implications.

Software Security Is Everyone's Job

Connectivity and distributed computation is so pervasive that the only way to begin to secure our computing infrastructure is to enlist everyone.

- Builders must practice security engineering, ensuring that the systems we build are defensible and not riddled with holes (especially when it comes to the software).
- Operations people must continue to architect reasonable networks, defend them, and keep them up.
- Administrators must understand the distributed nature of modern systems and begin to practice the principle of least privilege.

- Users must understand that software can be secure so that they can take their business to software providers who share their values. (Witness the rise of Firefox.) Users must also understand that they are the last bastion of defense in any security design and that they need to make tradeoffs for better security.
- Executives must understand how early investment in security design and security analysis affects the degree to which users will trust their products.

The most important people to enlist for near-term progress in computer security are the builders. Only by pushing past the standard-issue operations view of security will we begin to make systems that can stand up under attack